

Week 4 - Task 1

Understanding YOLOv26 Dataset Configuration Files

1. Introduction:

Object detection models such as YOLO (You Only Look Once) rely on well-structured datasets and configuration (meta) files for effective training and evaluation. These configuration files define how the dataset is organized, where the data is located, and how annotations are interpreted. In this task, I explored how datasets are structured and configured for YOLO-based object detection models, specifically YOLOv26. Through this process, I understood that training a model is not just about providing images, but also about organizing data correctly and defining it through configuration (meta) files. These files guide the model in locating the dataset, interpreting labels, and understanding the different object classes. The key components I analyzed include the dataset YAML file, directory structure, and label annotation format.

2. Dataset YAML File:

The dataset YAML file (commonly named data.yaml) serves as the central configuration file that informs the YOLO model about the dataset. It contains key information such as dataset paths, number of classes, and class names.

A typical YAML file includes:

- path: Root directory of the dataset
- train: Path to training images
- val: Path to validation images
- names: List of class names

Example:

```
path: /Users/ananya/Desktop/IIITH_Int/dataset  
  
train: images/train  
val: images/val
```

```
names:
  0: person
  1: car
  2: dog
  3: motorcycle
  4: skateboard
  5: cellphone
  6: umbrella
  7: bench
```

This file allows the model to locate the dataset and understand how many categories it needs to predict. The names field maps numerical class IDs (used in label files) to meaningful class labels. Without this configuration, the model cannot correctly interpret the dataset.

3. Directory Structure:

YOLO datasets follow a standardized directory structure that separates images and their corresponding annotations or labels. A typical structure is:

```
dataset/
├── images/
│   ├── train/
│   └── val/
├── labels/
│   ├── train/
│   └── val/
```

- The images/ folder contains all input images.
- The labels/ folder contains annotation files corresponding to each image.
- The dataset is divided into training and validation subsets to evaluate model performance.

Each image file in the images/train or images/val directory has a corresponding label file in labels/train or labels/val with the same filename but a .txt extension. For example, images/train/img1.jpg and labels/train/img1.txt

This one-to-one correspondence ensures that the model can correctly associate each image with its annotations during training.

4. Label Annotation Format:

YOLO uses a simple and efficient text-based format for annotations. Each image has a corresponding .txt file containing object information. Each line in the file represents one object.

The format is:

<class_id> <x_center> <y_center> <width> <height>

Where:

- class_id: Integer representing the object class (starting from 0)
- x_center, y_center: Coordinates of the bounding box center
- width, height: Dimensions of the bounding box

All coordinate values are normalized between 0 and 1, relative to the image dimensions. For example:

0 0.5 0.5 0.2 0.3

This indicates:

- Object belongs to class 0 (e.g., "person")
- Bounding box is centered in the image
- Width is 20% and height is 30% of the image size

This normalized format makes the annotations resolution-independent and efficient for training.

5. Conclusion:

Through this task, I gained a clear understanding of how YOLOv26 datasets are structured and configured. I learned that the YAML file acts as a blueprint for the dataset, the directory structure ensures proper organization, and the label files provide precise object annotations. From my perspective, these configuration files are fundamental because even a well-trained model cannot perform correctly if the dataset is not properly

defined. This exercise helped me connect the practical workflow of preparing data with the theoretical aspects of object detection, making the overall training process much clearer to me.

6. References:

- <https://www.ultralytics.com/glossary/image-segmentation>
- <https://www.kaggle.com/datasets/tommy011/data-yaml>
- <https://docs.ultralytics.com/#how-can-ultralytics-yolo-be-used-for-real-time-object-tracking>